

AUDIT LOGIQUE & FONCTIONNEL

ScolaGestion

V4

Rapport d'analyse approfondie de la plateforme SaaS de gestion scolaire : logique applicative, couverture fonctionnelle et intégrité des données. 162 modèles Prisma, 29 actions serveur, 18 modules d'interface et 8 tests d'intégrité exécutés en conditions réelles.

Plateforme : ScolaGestion V4 • SaaS de gestion scolaire
Audit : Z.ai • Analyse de complétude
2 septembre 2026

ANALYSE DE COMPLÉTUDE • SEPTEMBRE 2026

Table des matières

1. Synthèse exécutive	1
2. Méthodologie et périmètre	2
3. Acquis confirmés	3
4. Faille 1 — Sécurité et authentification inexistantes	3
5. Faille 2 — Intégrité des données : huit échecs prouvés	4
6. Faille 3 — Couverture fonctionnelle en vitrine	6
7. Failles secondaires et code mort	7
8. Référentiel de gestion scolaire complète	8
9. Plan de remédiation priorisé	9
10. Conclusion et verdict	10

1. Synthèse exécutive

Cette analyse constitue la troisième vague d'audit de ScolaGestion V4, et la première à porter exclusivement sur la **logique applicative et la couverture fonctionnelle** plutôt que sur la stabilité technique. Les vagues précédentes avaient corrigé vingt-sept erreurs TypeScript, cinq coquilles de schéma, l'idempotence du seed, vingt et une tables vides, l'erreur d'hydratation React, le portail élève et les notifications invisibles. Le socle technique est aujourd'hui sain : les trois serveurs répondent, la compilation est propre et la base de données ne contient plus une seule violation d'intégrité référentielle.

162	29	8/8	0
modèles Prisma au schéma	modèles en écriture (17,9 %)	tests d'intégrité en échec	contrôle d'authentification

Le présent audit révèle toutefois un écart d'une autre nature, structurel cette fois. Le système dispose d'un schéma de données remarquable par son ampleur — 162 modèles couvrant quatorze domaines de la gestion scolaire — mais l'application n'exploite réellement qu'une fraction de ce potentiel : seuls 29 modèles (17,9 %) sont jamais écrits par le code applicatif, 119 sont affichés en lecture seule et 14 ne sont référencés nulle part. Sept modules d'interface sur dix-huit sont des vitrines sans aucun flux de saisie, dont toute la chaîne RH, les services (cantine, transport, bibliothèque) et les deux portails familiaux.

Plus grave encore, la couche logique n'est pas blindée. Les vingt-neuf actions serveur du fichier unique `actions/index.ts` ne comportent ni validation d'entrée, ni contrôle d'autorisation, ni transaction, ni gestion d'erreur : aucun import de zod, aucun try/catch sur 646 lignes, aucun appel à `$transaction` dans tout le projet. Huit tests d'intégrité exécutés en conditions réelles contre la base de démonstration — en répliquant exactement le code des actions — se sont soldés par huit échecs, dont une perte d'écriture financière de 30 000 XOF sur deux encaissements concurrents et un stock passé à -150 unités.

Enfin, la question de la sécurité se pose en termes binaires : elle n'existe pas. Aucune page de connexion, aucun middleware, aucune session, aucune vérification de mot de passe ; les rôles et permissions du schéma ne sont jamais consultés par la moindre ligne de logique. Chaque action d'écriture est un point d'entrée HTTP public, l'identité de l'auteur est toujours fournie par le client, et des secrets (jetons de session, secrets 2FA, hachages de mots de passe) sont sérialisés dans le payload envoyé au navigateur. Le verdict global est donc nuancé : **plateforme de démonstration riche et visuellement aboutie, mais pas encore un système de gestion scolaire opérationnel.**

Verdict global : plateforme de démonstration riche (schéma exceptionnel de 162 modèles, base intègre, interface complète, zéro erreur technique) mais **non opérationnelle** : sécurité inexistante, intégrité des données non garantie (8/8 tests en échec), et 73,5 % du schéma exploité en lecture seule.

2. Méthodologie et périmètre

L'analyse a été conduite selon trois axes complémentaires, de manière à croiser les points de vue statique, dynamique et fonctionnel. Le premier axe est un croisement exhaustif entre le schéma de données et le code applicatif : les 162 noms de modèles extraits de `prisma/schema.prisma` (2 810 lignes) ont été recherchés systématiquement dans les 4 983 lignes du code source, en distinguant les usages en lecture (`findMany`, `findFirst`, `include`) des usages en écriture (`create`, `update`, `upsert`). Chaque cas ambigu a été tranché par lecture directe des lignes de code concernées, et la somme des catégories a été vérifiée : $29 + 119 + 14 = 162$.

Le deuxième axe est un audit de la couche logique proprement dite : lecture intégrale du fichier `src/app/actions/index.ts` (646 lignes, 29 actions exportées), de `src/app/page.tsx` (385 lignes, environ 130 requêtes Prisma dans un unique `Promise.all`) et de `src/components/app-shell.tsx`. Chaque action a été évaluée sur quatre critères : validation des entrées, contrôle d'autorisation, transactionnalité et gestion d'erreur. Les dix-huit modules d'interface ont été parcourus pour distinguer les interactions réelles (formulaires reliés à une action serveur) des interactions mortes (boutons sans gestionnaire, imports inutilisés, affirmations non implémentées).

Le troisième axe est expérimental : un script de test a été exécuté directement contre la base de démonstration (`db/custom.db`) en répliquant à l'identique le code des actions serveur concernées, afin de prouver en conditions réelles les défauts suspectés à la lecture. Huit scénarios ont été joués, dont deux en exécution concurrente (`Promise.all`) pour démontrer les courses critiques. Toutes les données de test ont ensuite été intégralement purgées ; l'intégrité de la base a été revalidée après nettoyage : 163/163 tables remplies, 0 violation de clé étrangère, 0 orphelin.

Source	Volume	Rôle dans l'audit
<code>prisma/schema.prisma</code>	2 810 lignes / 162 modèles	Cartographie des domaines, contraintes, cascades
<code>src/app/actions/index.ts</code>	646 lignes / 29 actions	Audit logique : validation, autorisation, transactions
<code>src/app/page.tsx</code>	385 lignes / ~130 requêtes	Contrôle du chargement de données et des fuites
<code>src/components/modules/</code> (18 fichiers)	4 710 lignes au total	Fonctionnalité réelle vs vitrine
<code>scripts/seed.ts</code>	1 521 lignes	Cohérence des données de démonstration
<code>scripts/test-failles-logique.ts</code>	8 scénarios	Preuves dynamiques (T1 à T8)

Tableau 1 — Sources auditées et leur contribution au présent rapport

3. Acquis confirmés

Avant d'exposer les failles, cet audit confirme solidement l'acquis des vagues précédentes. L'infrastructure est stable et complète : le serveur de développement (port 3000), le serveur de production reconstruit pour l'occasion (port 3100) et la passerelle (port 81) répondent tous en HTTP 200. La compilation TypeScript est strictement propre (zéro erreur, validation bloquante au build depuis le retrait d'ignoreBuildErrors) et le build de production s'exécute sans avertissement.

La base de données est saine et dense : les 163 tables issues du schéma sont toutes remplies par un seed désormais idempotent, l'intégrité référentielle est vérifiée sans aucune violation, et le contrôle des orphelins (élèves sans classe, utilisateurs sans école) est négatif. La couche présentation est complète : quinze modules navigables sans erreur JavaScript, cinq portails définis, zéro erreur d'hydratation depuis le correctif de fuseau horaire.

Sur le plan logique, plusieurs mécanismes bien construits méritent d'être crédités, car ils formeront des fondations solides pour la remédiation. La génération de bulletins calcule correctement les moyennes pondérées (ramène chaque note sur 20 selon le barème, pondère par le coefficient de l'évaluation puis par le coefficient de la matière, exclut les absents). L'encaissement des paiements applique une allocation FIFO raisonnée sur les échéances avec suivi de l'imputation via la table de jointure. Le workflow des bulletins à six états et la matrice RBAC affichée dans le module Sécurité traduisent une intention d'architecture correcte, même si leur application réelle reste à câbler.

4. Faille 1 — Sécurité et authentification inexistantes

Le constat central de cette faille tient en une phrase : **rien dans l'application ne permet d'identifier qui effectue une action**. Une recherche exhaustive des motifs login, signIn, session, cookie, middleware, bcrypt, jwt et password dans tout le code source ne retourne qu'un seul résultat : une ligne de texte descriptif dans le module vitrine. Il n'existe ni page de connexion, ni formulaire d'authentification, ni middleware de protection de route, ni lecture de cookie de session côté serveur. Le changement de portail (Super-Admin, Direction, Enseignant, Parent, Élève) est un simple menu déroulant d'état client, explicitement étiqueté « démo » dans le code, et le filtrage des modules par portail est purement cosmétique : il ne repose que sur un useState dans le shell, sans aucun garde-fou serveur.

Conséquence directe : les vingt-neuf actions serveur marquées « use server » sont des points d'entrée HTTP invocables par n'importe quel visiteur, y compris hors de l'interface. Plus préoccupant pour la valeur probante du journal d'audit : l'identité de l'auteur est systématiquement fournie par le client. Les champs saisiParId, encaisseParId, creeParId, declareParId, decideParId et valideeParId proviennent de champs de formulaire — parfois même de champs texte visibles et modifiables — ce qui rend la falsification du journal d'audit triviale. Dans le portail enseignant, toutes les écritures sont attribuées à la direction ; dans le portail élève, le « moi » est simplement le premier élève de la liste.

La couche de sécurité du schéma existe pourtant intégralement — Utilisateur avec motDePasseHash et verrouillage après échecs, SessionUtilisateur, TwoFactorMethod, JetonAuth, TentativeConnexion, ApiToken, Role, Permission, RolePermission, UtilisateurRole — et toutes ces tables sont remplies par le seed. Mais aucune ligne de logique ne les consulte jamais : les hachages insérés sont des constantes factices, aucun mot de passe ne pourrait être vérifié, et les dépendances next-auth et zod présentes dans package.json ne sont importées nulle part. Le RBAC est affiché, jamais appliqué.

Deux aggravations doivent être signalées. D'abord, l'isolation multi-tenant est absente : la plupart des actions écrivent sur l'école de démonstration codée en dur (slug « vinci »), mais aucun identifiant entrant — élève, évaluation, séance, bulletin, article, dépense, incident — n'est vérifié comme appartenant à cette école ; des écritures inter-établissements par identifiants forgés sont possibles, et l'action changerStatutEcole permet à tout visiteur de suspendre n'importe quelle école cliente. Ensuite, une fuite de secrets : page.tsx sérialise sessionsUtilisateur (tokenHash), twoFactorMethods (secret), jetonsAuth, apiTokens et la table utilisateurs complète (motDePasseHash) dans les props du composant client — donc dans le payload RSC transmis au navigateur, en plus des données sensibles élèves (besoins spécifiques, santé, signalements mineurs, bulletins de paie).

Exigence minimale	État constaté	Localisation
Authentification (connexion, session)	Absente — aucun login, aucun middleware, aucun cookie lu	src/app/ entier (0 occurrence)
Autorisation (rôle / permission par action)	Absente — 29 actions publiques, rôle jamais vérifié	actions/index.ts 1.1-646
Identité de l'auteur contrôlée côté serveur	Fournie par le client (champs texte ou cachés)	finances.tsx 1.77, pedagogique.tsx 1.63/139
Isolation multi-tenant des IDs entrants	Absente — école « vinci » codée en dur, IDs non vérifiés	actions/index.ts 1.9
Non-exposition des secrets	tokenHash, secrets 2FA, motDePasseHash dans le payload client	page.tsx 1.204-209 et 369-370

Tableau 2 — Exigences de sécurité minimales et état vérifié

5. Faille 2 — Intégrité des données : huit échecs prouvés

Pour objectiver les défauts de logique révélés par la lecture du code, huit scénarios de test ont été exécutés contre la base de démonstration en répliquant à l'identique les instructions des actions serveur concernées (mêmes requêtes Prisma, même ordre d'opérations, mêmes formats de données). Chaque scénario simule un geste utilisateur ordinaire : un double-clic, deux guichets qui encaissent en parallèle, une faute de frappe dans un type de mouvement, une note saisie au delà du barème. Les huit tests ont échoué, c'est-à-dire que chacun a produit un état de base incohérent, une erreur non gérée, ou les deux.

Les causes racines sont au nombre de trois et sont systémiques. Premièrement, **aucune validation d'entrée** : les valeurs issues de FormData sont converties par String() et Number() sans aucun contrôle, si bien qu'un champ absent devient la chaîne « null », qu'un montant peut être négatif ou NaN, et qu'une date invalide provoque un crash Prisma non intercepté. Deuxièmement, **aucune transaction** : les opérations multi-tables (paiement puis allocation d'échéances, mouvement de stock puis mise à jour d'article, lecture de version puis création de bulletin) sont exécutées en séquence non atomique, ouvrant des courses critiques. Troisièmement, **aucune gestion d'erreur** : zéro try/catch sur 646 lignes, et les violations de contrainte d'unicité (code Prisma P2002) remontent comme des erreurs serveur brutes au lieu d'un message exploitable.

Test	Scénario (code de l'action répliqué)	Constat vérifié en base	Gravité
T1 — Régénération d'échéances	genererEcheancesClasse exécuté deux fois (double-clic)	8 échéances créées pour 4 élèves : doublons purs, aucun @@unique sur EcheanceFrais	Majeure
T2 — Paiements concurrents	Deux encaissements de 30 000 XOF en parallèle sur une échéance de 50 000 (sans \$transaction)	60 000 encaissés, échéance soldée à 30 000 : 30 000 XOF non tracés (écriture perdue)	Critique
T3 — Mouvement de stock	Sortie de 350 pour un stock de 250, puis type « entrée » accentué	Stock passé de 250 à -100, puis à -150 : négatif autorisé, tout type autre que « entree » décrémente	Majeure
T4 — Note hors barème	Saisie d'une note de 25 sur un barème /20 (saisirNotes)	Note 25 acceptée et comptée telle quelle dans le calcul de moyenne	Majeure
T5 — Course de version bulletin	Deux générations simultanées du même bulletin (genererBulletin)	Erreur P2002 brute sur (eleveId, periodeId, version) : lecture-incrémentation non atomique	Majeure
T6 — Montant négatif	Encaissement de -50 000 XOF (encaisserPaiement)	Paiement négatif persisté : recettes faussées, aucun contrôle de signe	Critique
T7 — Matricule collisionnel	Format EL- + 6 derniers chiffres de Date.now() (inscrireEleve)	Deux appels dans la même milliseconde donnent le même matricule (vérifié) ; P2002 non géré	Majeure
T8 — Sortie de mineur	Sortie anticipée sans autorisation parentale (sortieEleve)	Sortie auto-validée (validationExceptionnelle=true), parentsNotifies=true sans aucune notification créée	Critique

Tableau 3 — Résultats des huit tests d'intégrité exécutés en base (réplication exacte du code des actions)

Il convient de souligner la portée financière et juridique de deux de ces échecs. Le test T2 démontre qu'une école utilisant deux postes de guichet simultanément verrait des encaissements disparaître de la comptabilité élève sans aucune trace d'anomalie — le paiement existe, son imputation existe, mais l'échéance n'en conserve pas la mémoire. Le test T8, quant à lui, simule la sortie d'un enfant récupéré par une personne non autorisée : le système l'enregistre comme validée par exception et prétend avoir notifié les parents, alors qu'aucune notification n'est créée. Dans un contexte réel, ces deux comportements exposeraient l'établissement à des litiges financiers et à un risque de sûreté des mineurs.

8/8	30 000 XOF	-150	0
-----	------------	------	---

tests en échec	perdus sur 60 000 (T2)	unités de stock atteintes (T3)	notification créée (T8)
----------------	------------------------	--------------------------------	-------------------------

6. Faille 3 — Couverture fonctionnelle en vitrine

Le croisement exhaustif entre les 162 modèles du schéma et le code applicatif aboutit au bilan suivant : 29 modèles sont réellement écrits (17,9 %), 119 sont consultés en lecture sans jamais être modifiés par l'application (73,5 %), et 14 ne sont référencés dans aucune ligne de code (8,6 %) — leurs données seedées sont définitivement invisibles. Deux modèles présentent un cas particulièrement révélateur : Abonnement (l'état d'abonnement SaaS d'une école) et PaiementEcheance (l'imputation des paiements sur les échéances) sont écrits mais jamais relus — l'application fabrique de l'état qu'elle ne sait pas consulter.

À l'échelle de l'interface, le constat est cohérent : onze modules sur dix-huit offrent au moins un flux d'écriture, mais trois d'entre eux n'en ont qu'un seul (présences, examens, rendez-vous), et sept sont des vitrines pures sans aucune écriture : le tableau de bord direction (par design), le module RH, le module services, l'audit, le portail parent, le portail élève et le méta-module « V4 » dont les trente-sept sections sont trente-sept tables en lecture seule. Environ trente-quatre ensembles de données supplémentaires sont chargés à chaque requête de page sans jamais être affichés par le moindre composant : budgets détaillés, chaîne fournisseurs (commandes, lignes, paiements), candidatures RH, conventions de stage, appareils mobiles, tentatives de connexion, journaux de webhooks, consentements de communication — un poids mort servi à chaque rendu.

Surtout, plusieurs briques attendues d'une gestion scolaire complète manquent à l'appel, alors même que les tables correspondantes existent et sont remplies : l'éditeur d'emploi du temps avec détection de conflits (aucune recherche de chevauchement prof/salle/classe dans tout le code), le module santé-infirmerie (aucun modèle dédié : seuls les champs allergies et condition médicale de l'élève existent), la saisie des résultats d'examens officiels (l'action serveur existe mais aucune interface ne l'appelle), les convocations, les reçus de paiement imprimables, la réinscription annuelle et la transition d'année scolaire (archivage des classes, bascule des élèves). Les bulletins, cœur du système, restent incomplets : pas de rang, pas d'appréciation, pas de mention, pas d'impression — et le panneau de détail par matière est inatteignable car la fonction `setSelectedBulletinId` n'est jamais appelée.

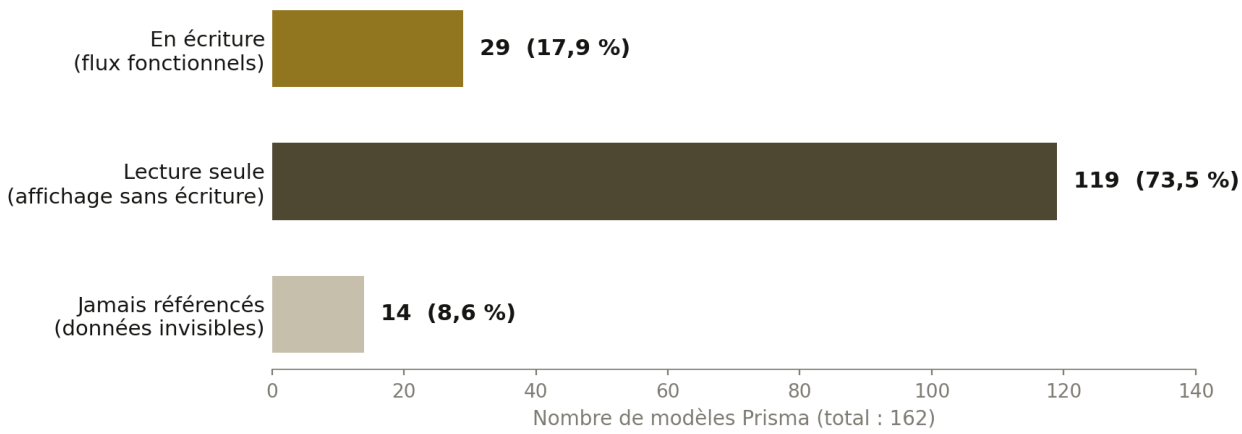


Figure 1 — Répartition des 162 modèles Prisma selon leur usage dans le code applicatif

Catégorie de modèles	Exemples représentatifs	Conséquence
En écriture (29)	Eleve, Note, Bulletin, Paiement, EcheanceFrais, Presence, Incident, Sanction, Salle	Flux fonctionnels réels — le cœur opérationnel de la démo
Lecture seule (119)	Conge, Remplacement, BulletinPaie, OffreEmploi, Budget, Conversation, CantineInscription, BiblioPret, EmploiTemps, ReservationSalle	Données seedées affichées sans aucun flux de saisie : RH, compta, services, messagerie
Jamais référencés (14)	EleveParent, VoteConseil, Chapitre, ReceptionCommande, SuiviSignalement, TicketStatutHistorique, PushToken, WidgetDashboard	Lien élève-parent, délibérations, réceptions de commandes : invisibles par construction
Écrits mais jamais lus (2)	Abonnement, PaiementEcheance	État fabriqué non consulté : la source de vérité SaaS et l'imputation ne sont jamais affichées

Tableau 4 — Catégories de modèles et leurs conséquences fonctionnelles

7. Failles secondaires et code mort

Au-delà des trois failles majeures, l'inventaire des défauts de moindre gravité — mais répétitifs — éclaire la qualité d'ensemble de la couche logique. Ils sont regroupés en cinq familles : code mort, promesses d'interface non tenues, non-déterminisme, générateurs d'identifiants faibles, et défauts d'architecture de chargement. Chaque item a été vérifié par lecture directe du fichier et de la ligne concernés.

- **Code mort confirmé** — L'action `saisirResultatExamen` (`actions/index.ts` 1.515) n'est appelée par aucune interface ; `setSelectedBulletinId` (`pedagogique.tsx` 1.37) n'est jamais invoquée, rendant le détail des bulletins inatteignable ; les imports `actions`, `useTransition`, `ModalForm` et `CreateButton` de `services.tsx`, `personnel.tsx`, `parent-portal.tsx` et `examens.tsx` sont inutilisés — des modules qui « paraissent » interactifs sans l'être.

- **Données saisies puis jetées** — Le champ « motif » de l'appel de présences (presences.tsx l.115) est soumis avec le formulaire mais ignoré par saisirAppel, qui ne filtre que les clés presence_ : le motif d'absence saisi par l'enseignant n'arrive jamais en base.
- **Notifications factices** — Les indicateurs notifieParents et parentsNotifies sont écrits à true en dur sans qu'aucune notification ne soit créée (sanctionner l.442, sortieEleve l.577) ; aucune messagerie SMS, e-mail ou push n'a de véritable envoi ; le rappel de rendez-vous « 24 h avant » et le mode « hors-ligne IndexedDB » de la fiche de présences sont des affirmations d'interface sans code sous-jacent.
- **Non-déterminisme** — getDirectionUserId (actions l.15-18) choisit un utilisateur « personnel » sans tri : n'importe quel enseignant peut devenir l'auteur attribué des écritures ; la direction affichée, l'année active et le super-admin sont sélectionnés par findFirst sans orderBy (page.tsx l.30, l.35-39, l.104).
- **Identifiants collisionnels** — Matricules EL- à 6 chiffres d'horloge (collision entre inscriptions espacées de 16,7 min exactement, et double-clic simultané vérifié), références PAY- à 8 chiffres, badges visiteur V- à 4 chiffres aléatoires (Math.random) — aucun compteur séquentiel, aucune gestion de collision.
- **Suppressions et cycle de vie absents** — Aucun delete dans tout le code applicatif : impossible de corriger une saisie erronée, de retirer un élève, d'annuler un paiement ; aucune cascade onDelete dans le schéma (0 occurrence) ; les outils RGPD (demande d'effacement, export) sont des vitrines — incompatibles avec les obligations affichées.
- **Architecture de chargement** — Environ 130 requêtes dans un unique Promise.all à chaque rendu, sans pagination ni cache (seul auditLog est plafonné à 100 lignes) ; le client Prisma journalise toutes les requêtes en production ; l'unique route API est un stub « Hello, world! » ; les dépendances next-auth et zod sont installées et jamais importées.

8. Référentiel de gestion scolaire complète

La matrice ci-dessous confronte l'application au référentiel usuel d'une gestion scolaire complète, domaine par domaine. Quatre niveaux sont distingués : « Fonctionnel » signifie qu'au moins un flux d'écriture complet existe et persiste ; « Partiel » qu'un flux existe mais avec des lacunes de logique ou d'interface ; « Vitrine » que les données existent et s'affichent sans aucune saisie possible ; « Absent » qu'aucune brique dédiée n'existe. Les vérifications d'écriture proviennent du croisement exhaustif modèles-code de l'axe 1.

Domaine	Niveau	Commentaire vérifié
Scolarité (inscription, classes, historique)	Partiel	Inscription fonctionnelle avec matricule collisionnel ; réinscription, transfert et transition d'année absents
Évaluation et bulletins	Partiel	Saisie de notes et moyennes pondérées correctes ; workflow à 6 états sans machine à états ; rang, mention et impression absents ; détail inatteignable

Domaine	Niveau	Commentaire vérifié
Présences et vie scolaire	Partiel	Appel quotidien fonctionnel (motif perdu) ; justificatifs d'absence en lecture seule ; incidents et sanctions fonctionnels (notifications factices)
Finances élèves (frais, échéances, encaissements)	Partiel	FIFO bien conçue mais non transactionnelle ; montants non validés ; échéances dupliquables ; reçus absents
Comptabilité générale et achats	Vitrine	Budgets, écritures, journaux, fournisseurs, commandes : 100 % lecture seule
Ressources humaines	Vitrine	Congés, remplacements, paie, évaluation, recrutement : aucun flux d'écriture
Services (cantine, transport, bibliothèque)	Vitrine	Inscriptions et prêts seedés ; ni scanner cantine, ni prêt-retour, ni attribution de manuels
Santé et infirmerie	Absent	Aucun module ni modèle dédié ; seuls champs allergies et condition médicale de l'élève
Emploi du temps	Vitrine	Séances seedées sans éditeur ; aucune détection de conflit prof/salle/classe dans le code
Communication et portails	Partiel	Notifications internes fonctionnelles ; SMS/e-mail/push jamais envoyés ; portails parent et élève en vitrine (élève n°1 codé en dur)
Examens officiels	Partiel	Inscription de masse idempotente ; saisie des résultats sans interface ; convocations et délibérations en vitrine
Sécurité physique du site	Partiel	Visiteurs et sorties fonctionnels ; sortie sans autorisation auto-validée ; pas de sortie de visiteur ; habilitations en vitrine
Couche SaaS (plans, écoles, facturation)	Partiel	Création d'école et suspension fonctionnelles mais publiques ; abonnement jamais lu ; facturation périodique absente
Conformité et RGPD	Vitrine	Registres, consentements, effacement et export : données affichées, aucun mécanisme réel (0 delete dans l'app)

Tableau 5 — Matrice de complétude par domaine (niveaux : Fonctionnel, Partiel, Vitrine, Absent)

9. Plan de remédiation priorisé

Le plan ci-dessous ordonne les corrections par priorité décroissante de risque, en s'appuyant sur l'existant : la couche de sécurité du schéma existe déjà et est remplie, zod et next-auth sont déjà des dépendances installées, et la matrice RBAC est déjà affichée — la remédiation P0 consiste donc pour l'essentiel à brancher des fondations prêtes plutôt qu'à construire. Les estimations d'effort supposent le contexte actuel (une action, une page, un seed) et une seule personne au développement.

Priorité	Correctifs	Effort
P0 — Bloquant avant toute mise en service	Authentification réelle (NextAuth, déjà en dépendance) branchée sur Utilisateur ; vérification de session et de rôle dans chaque action via le RBAC existant ; \$transaction sur encaisserPaiement, genererBulletin, enregistrerMouvementStock, genererEcheancesClasse ; validation zod de tous les FormData ; purge des secrets et des ~34 datasets invisibles du payload ; vérification d'appartenance tenant de chaque ID entrant	2 à 3 semaines
P1 — Correction de la logique métier	Bornage des notes (0 à barème) et des montants (positifs) ; idempotence des échéances (@@unique eleve-frais-date + upsert) ; matricule séquentiel ; machine à états stricte pour les bulletins avec rang, mention et appréciation ; garde-fou stock négatif et types de mouvement contrôlés ; persistance du motif d'appel ; création de notifications réelles (sanctions, sorties) ; branchement du code mort (détail bulletins, interface de saisie des résultats d'examens) ; try/catch systématique avec retours exploitables	2 semaines
P2 — Complétude fonctionnelle	Éditeur d'emploi du temps avec détection de conflits ; module santé-infirmier ; flux RH (demande et validation de congés, remplacements) ; flux cantine, bibliothèque et manuels ; portails parent et élève réels après connexion ; reçus de paiement imprimables ; convocations aux examens ; transition d'année scolaire	4 à 6 semaines
P3 — Hygiène et performance	Suppression du code mort et des dépendances fantômes ; pagination et fractionnement du Promise.all de page.tsx ; journalisation des requêtes limitée au développement ; vrai point d'entrée API ou suppression du stub ; findFirst avec orderBy déterministes partout	3 à 4 jours

Tableau 6 — Correctifs ordonnés par risque décroissant, avec effort estimé

10. Conclusion et verdict

Réponse directe aux trois questions posées. **La logique est-elle complète ?** Non : la couche d'actions est une maquette fonctionnelle — les intentions sont bonnes (FIFO des paiements, moyennes pondérées, workflow de bulletins) mais l'absence systémique de validation, de transaction, de contrôle d'autorisation et de gestion d'erreur la rend impropre à un usage réel, ce que les huit tests d'intégrité ont démontré sans exception.

Tous les éléments d'une gestion scolaire complète sont-ils présents ? Au niveau des données, presque : le schéma couvre quatorze domaines avec une finesse rare, y compris la conformité RGPD et la sûreté des mineurs. Au niveau de l'application, non : moins d'un modèle sur cinq est écrit, la chaîne RH, la comptabilité, les services et les portails familiaux sont des vitrines, et l'emploi du temps, la santé, les résultats d'examens et les reçus n'ont aucun flux. **Reste-t-il des failles non résolues ?** Oui, et elles sont désormais toutes documentées, localisées fichier par fichier et ligne par ligne, avec un plan de remédiation chiffré : trois failles majeures (sécurité inexistante, intégrité non garantie, couverture en vitrine) et cinq familles de défauts secondaires.

La trajectoire de correction est toutefois favorable : le socle technique est sain, la base est intègre, le schéma est prêt pour la sécurité (sessions, 2FA, RBAC remplis) et les dépendances nécessaires sont déjà installées. La priorité absolue est le lot P0 : sans authentification et sans transactions, aucune autre correction n'a de valeur opérationnelle. Une fois ce lot livré, le lot P1 peut être conduit action par action avec les tests du présent rapport comme critères d'acceptation — les huit scénarios T1 à T8 constituent précisément la suite

de non-régression idéale pour valider la remédiation.

Prochaine étape recommandée : engager le lot P0 (authentification NextAuth branchée sur le RBAC seedé, \$transaction sur les quatre flux concurrents, validation zod, purge des secrets). Les huit scénarios T1 à T8 du présent rapport constituent la suite de non-régression naturelle pour valider chaque correctif.